PRACTICAL 6

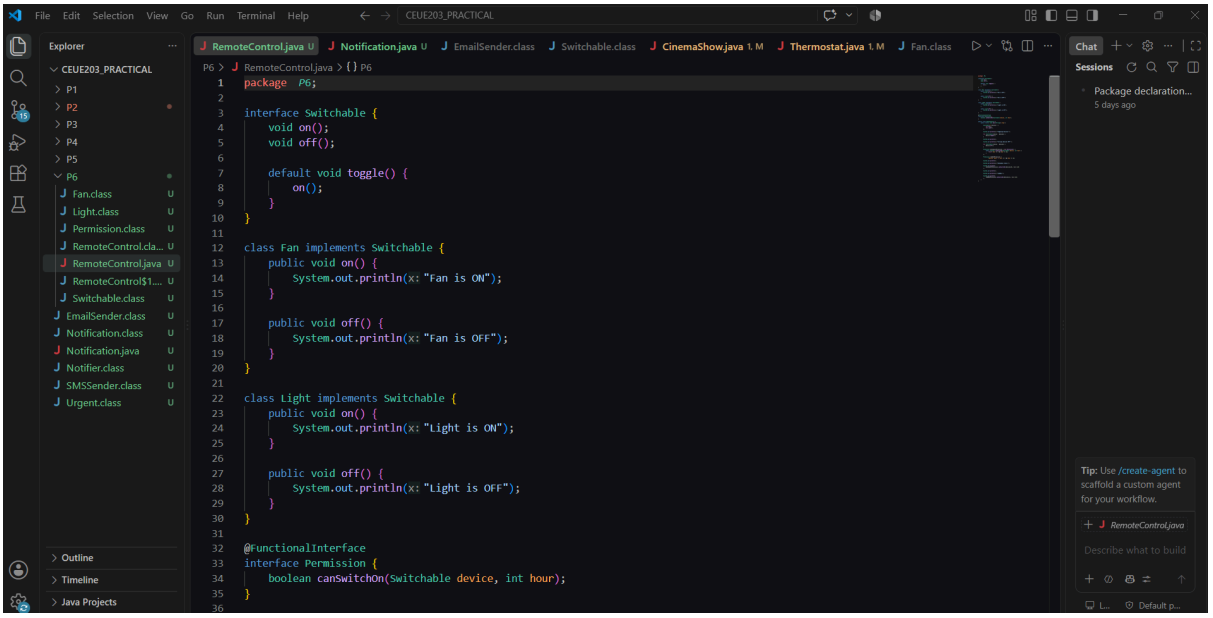
1. Remote control: an interface Switchable with on()/off() and a default toggle(); Fan and

Light implement it. Loop over a Switchable[] toggling each. Then add a functional

interface that decides, given a device and an hour, whether it may switch on, and supply it

once as an anonymous class and once as a lambda..

CODE :



```java
package P6;

interface Switchable {
    void on();
    void off();

    default void toggle() {
        on();
    }
}

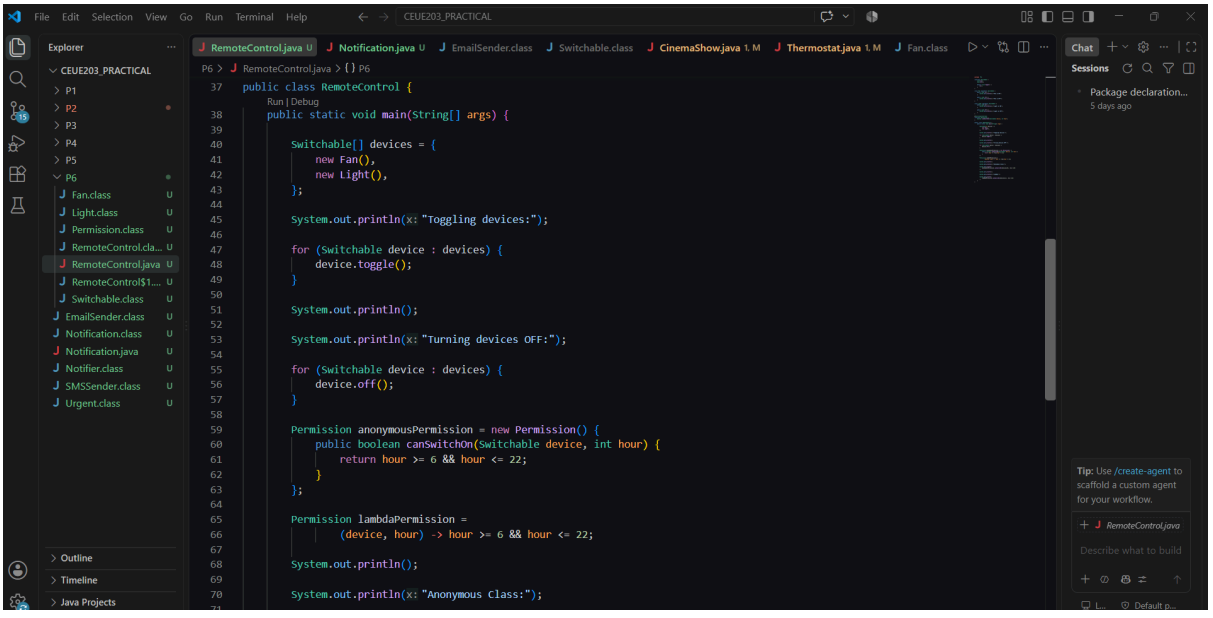
class Fan implements Switchable {
    public void on() {
        System.out.println(x: "Fan is ON");
    }

    public void off() {
        System.out.println(x: "Fan is OFF");
    }
}

class Light implements Switchable {
    public void on() {
        System.out.println(x: "Light is ON");
    }

    public void off() {
        System.out.println(x: "Light is OFF");
    }
}

@FunctionalInterface
interface Permission {
    boolean canSwitchOn(Switchable device, int hour);
}
```



```java
public class RemoteControl {
    Run | Debug
    public static void main(String[] args) {

        Switchable[] devices = {
            new Fan(),
            new Light(),
        };

        System.out.println(x: "Toggling devices:");

        for (Switchable device : devices) {
            device.toggle();
        }

        System.out.println();

        System.out.println(x: "Turning devices OFF:");

        for (Switchable device : devices) {
            device.off();
        }

        Permission anonymousPermission = new Permission() {
            public boolean canSwitchOn(Switchable device, int hour) {
                return hour >= 6 && hour <= 22;
            }
        };

        Permission lambdaPermission =
                (device, hour) -> hour >= 6 && hour <= 22;

        System.out.println();

        System.out.println(x: "Anonymous Class:");
```



```
public class RemoteControl {
    public static void main(String[] args) {

        System.out.println(x: "Turning devices OFF:");

        for (Switchable device : devices) {
            device.off();
        }

        Permission anonymousPermission = new Permission() {
            public boolean canSwitchOn(Switchable device, int hour) {
                return hour >= 6 && hour <= 22;
            }
        };

        Permission lambdaPermission =
                (device, hour) -> hour >= 6 && hour <= 22;

        System.out.println();

        System.out.println(x: "Anonymous Class:");

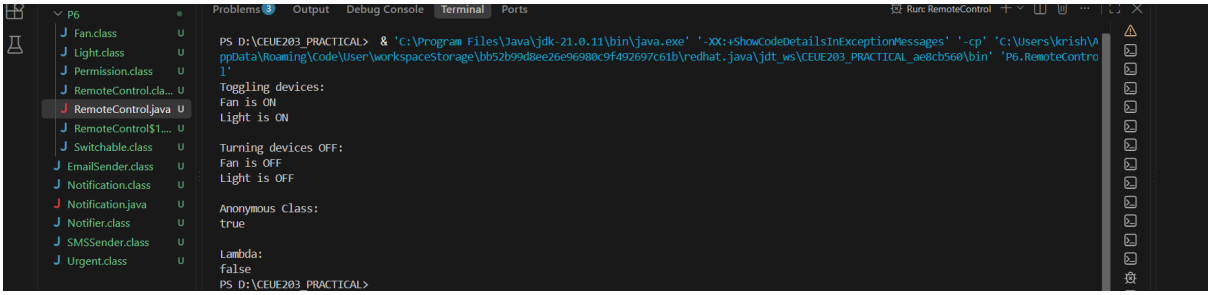
        System.out.println(
            anonymousPermission.canSwitchOn(devices[0], hour: 10)
        );

        System.out.println();

        System.out.println(x: "Lambda:");

        System.out.println(
            lambdaPermission.canSwitchOn(devices[1], hour: 23)
        );
    }
}
```

OUTPUT :



```
PS D:\CEUE203_PRACTICAL> & 'C:\Program Files\Java\jdk-21.0.11\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\krish\AppData\Roaming\Code\User\workspaceStorage\bb52b99d8ee26e96980c9f492697c61b\redhat.java\jdt_ws\CEUE203_PRACTICAL_ae8cb560\bin' 'P6.RemoteControl'
Toggling devices:
Fan is ON
Light is ON

Turning devices OFF:
Fan is OFF
Light is OFF

Anonymous Class:
true

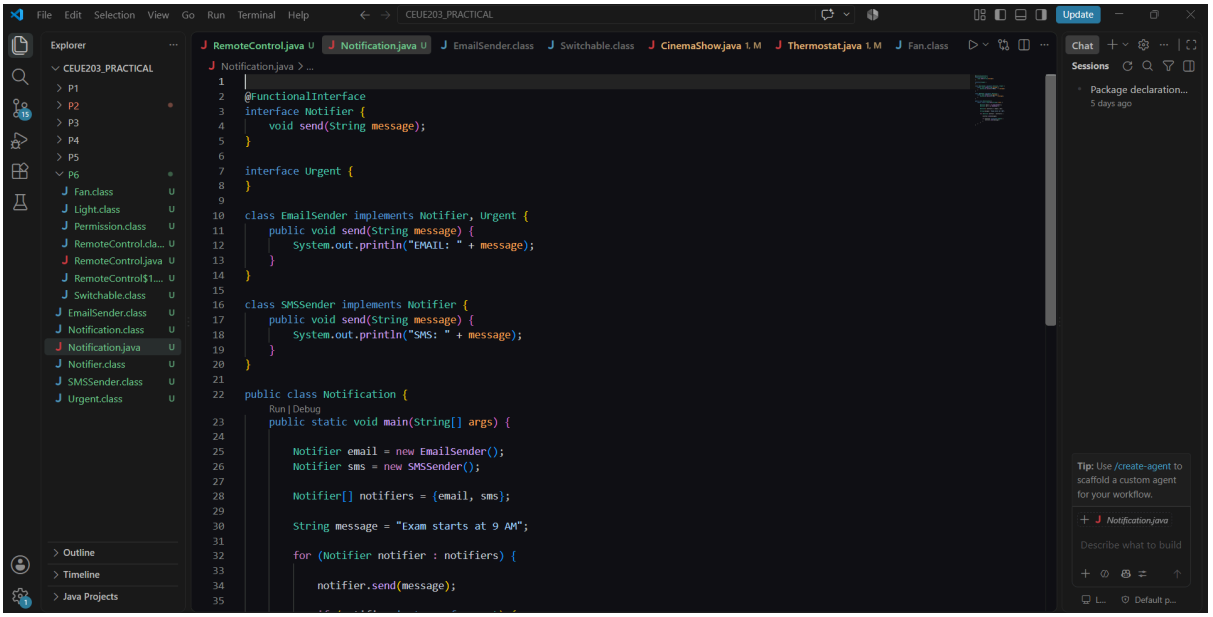
Lambda:
false
PS D:\CEUE203_PRACTICAL>
```

2. Notification senders: a functional interface Notifier with send(String); make email and

SMS senders as lambdas, hold them in an array, and broadcast one message through all.

Add a marker interface Urgent and send twice for senders marked urgent.

CODE :



```java
@FunctionalInterface
interface Notifier {
    void send(String message);
}

interface Urgent {
}

class EmailSender implements Notifier, Urgent {
    public void send(String message) {
        System.out.println("EMAIL: " + message);
    }
}

class SMSSender implements Notifier {
    public void send(String message) {
        System.out.println("SMS: " + message);
    }
}

public class Notification {
    Run | Debug
    public static void main(String[] args) {

        Notifier email = new EmailSender();
        Notifier sms = new SMSSender();

        Notifier[] notifiers = {email, sms};

        String message = "Exam starts at 9 AM";

        for (Notifier notifier : notifiers) {

            notifier.send(message);
```



```
PS D:\CEUE203_PRACTICAL\P6> cd "d:\CEUE203_PRACTICAL\" ; if ($?) { javac Notification.java } ; if ($?) { java Notification }
EMAIL: Exam starts at 9 AM
EMAIL: Exam starts at 9 AM
SMS: Exam starts at 9 AM
PS D:\CEUE203_PRACTICAL>
```

KEY QUESTIONS :

1. What is the difference between an interface and an abstract class?

An interface is used to define rules that a class must follow.
An abstract class can have both abstract and normal methods.
A class can implement multiple interfaces but can extend only one abstract class.

2. What is a functional interface, and how does a lambda use it?

A functional interface has only **one abstract method**.
A lambda expression provides the implementation of that method.
It makes the code shorter and easier to write.

3. Why is a default method useful in an interface?

A default method has a **body** inside an interface.
The implementing class can use it directly without writing it again.
It helps add new functionality without changing existing classes.

LEARNING OUTCOME:

The learning outcome of this practical is to understand how interfaces work in Java and how different devices like Fan and Light can use the same interface. Students learn to use on(), off(), and default toggle() methods, store different devices in an interface array, and control them using a loop. They also learn about functional interfaces, anonymous classes, and lambda expressions to make decisions based on a device and time.